

PRUEBA PRÁCTICA

Gestión de Tienda Virtual con Microservicios, Inventario y Órdenes

Práctica de preparación técnica

Propósito: evaluar la capacidad del equipo para analizar, diseñar, desarrollar y desplegar una solución basada en microservicios, aplicando arquitectura Clean, API Gateway, SQL Server, caché nativa de .NET, patrón Unit of Work, pruebas unitarias, control de concurrencia, health checks, documentación OpenAPI y frontend con actualización sin refresco manual.

Documento Confidencial. Prohibida su reproducción total o parcial sin autorización.

1. Enunciado a resolver

Una empresa requiere implementar una tienda virtual mínima para comercializar productos en línea. La solución debe permitir consultar productos, agregarlos al carrito y confirmar órdenes, garantizando que no se venda más inventario del disponible, incluso cuando varios usuarios intenten comprar simultáneamente el mismo producto.

Cada producto cuenta con:

- Nombre del producto.
- Categoría.
- Precio unitario.
- Stock disponible.
- Estado: Activo / Inactivo.
- Fecha de creación.

Cada orden cuenta con:

- Cliente.
- Fecha de creación.
- Detalle de productos comprados.
- Cantidad por producto.
- Subtotal, impuesto y total.
- Estado: Pendiente / Confirmada / Cancelada.

Se requiere una solución tecnológica que permita gestionar el catálogo, carrito de compra y confirmación de órdenes, asegurando consistencia de inventario, trazabilidad y actualización visual sin necesidad de refrescar la página.

2. Objetivo de la Prueba

- Diseñar y desarrollar una solución para consultar productos disponibles en una tienda virtual.
- Agregar productos al carrito validando cantidades solicitadas.
- Confirmar una orden descontando stock de forma transaccional.
- Cancelar una orden mediante cambio lógico de estado y restauración de inventario cuando aplique.
- Evitar ventas superiores al stock disponible, incluso ante accesos concurrentes.
- Exponer la lógica de negocio mediante microservicios independientes comunicados por un API Gateway.
- Aplicar caché nativa de .NET en endpoints de lectura del catálogo.
- Aplicar el patrón Unit of Work en operaciones transaccionales de órdenes e inventario.
- Demostrar buenas prácticas de diseño, pruebas unitarias, manejo de errores y documentación técnica.

3. Modalidad de la Prueba

- Trabajo en equipo: 2 personas.
- Tiempo de duración: 6 horas.
- Se permite utilizar documentación oficial y herramientas de desarrollo autorizadas por el evaluador.
- No se permite apoyo externo ni colaboración con personas fuera del equipo asignado.

4. Alcance Funcional

4.1 Gestión de Productos y Catálogo

El microservicio de Catálogo debe permitir administrar y consultar productos. El catálogo debe estar disponible para el frontend a través del API Gateway. Los endpoints de consulta deben aplicar caché nativa de .NET para mejorar tiempos de respuesta en operaciones de lectura.

- Registrar productos indicando nombre, categoría, precio, stock inicial y estado.
- Consultar listado de productos activos.
- Consultar detalle de un producto.
- Editar precio, nombre, categoría y estado del producto.
- Consultar stock disponible del producto.
- Invalidar o actualizar la caché cuando se modifique un producto o su stock.

Campo requerido	Descripción
Nombre	Nombre comercial del producto.
Categoría	Categoría del producto.
Precio unitario	Precio base antes de impuestos.
Stock disponible	Cantidad disponible para venta.
Estado	Activo / Inactivo.
Fecha de creación	Fecha de registro del producto.

Reglas de negocio:

- No se pueden registrar productos con precio negativo o igual a cero.
- No se pueden registrar productos con stock negativo.
- No se deben mostrar productos inactivos en el catálogo público.
- El stock no debe ser actualizado únicamente desde el frontend.
- La caché debe usarse solo para lecturas del catálogo; no debe ser la fuente de verdad del inventario.
- La fuente de verdad para stock y órdenes debe ser SQL Server.

4.2 Carrito de Compra

El frontend debe permitir seleccionar productos y agregarlos a un carrito de compra simple. El carrito puede mantenerse en memoria del cliente, sesión o backend, siempre que al confirmar la orden se validen nuevamente las reglas de negocio en el backend.

- Agregar productos al carrito indicando producto y cantidad.
- Editar cantidades antes de confirmar la orden.
- Eliminar productos del carrito.
- Mostrar subtotal por línea, subtotal general, impuesto y total.
- Validar visualmente que la cantidad sea mayor a cero.
- Mostrar advertencia cuando la cantidad solicitada supere el stock consultado.

Reglas de negocio:

- No se permiten cantidades negativas o iguales a cero.
- No se permite agregar productos inactivos.
- Las validaciones del frontend no sustituyen las validaciones del backend.
- El precio final de la orden debe calcularse en backend al momento de confirmar.

4.3 Confirmación de Órdenes

El microservicio de Órdenes debe confirmar compras de forma transaccional. La operación de confirmación debe crear la orden, crear sus detalles, validar stock, descontar inventario y calcular totales como una única unidad de trabajo.

- Registrar una orden indicando cliente y productos comprados.
- Calcular subtotal, impuesto y total.
- Descontar stock de cada producto comprado.
- Persistir la orden y su detalle.
- Registrar la orden en estado Confirmada si todas las validaciones son exitosas.
- Rechazar la orden si algún producto no tiene stock suficiente.
- Aplicar patrón Unit of Work para confirmar o revertir toda la operación.

Cálculo	Regla
Subtotal línea	PrecioUnitario * Cantidad
Subtotal orden	Suma de subtotales de líneas
Impuesto	Subtotal * porcentaje de impuesto configurado
Total	Subtotal + impuesto
Stock resultante	StockActual - CantidadComprada

Reglas de negocio - Validaciones:

- No se puede confirmar una orden vacía.
- No se puede comprar un producto inactivo.
- No se puede comprar una cantidad mayor al stock disponible.
- No se permiten cantidades negativas o iguales a cero.
- El precio usado para calcular la orden debe venir desde backend.
- La operación debe ser atómica: si falla una línea, no se registra parcialmente la orden.
- El sistema debe manejar concurrencia adecuadamente. Cuando dos usuarios intenten comprar simultáneamente la última unidad de un producto, solo una orden debe completarse.

4.4 Cancelación de Órdenes

- La cancelación debe ser lógica; la orden cambia a estado Cancelada.
- Si la orden estaba Confirmada, debe restaurarse el stock de los productos correspondientes.
- No se permite cancelar dos veces la misma orden.
- No se debe eliminar físicamente la orden ni su detalle.
- Se mantiene trazabilidad de fecha de cancelación y motivo.

4.5 Dashboard de Tienda

Se requiere un dashboard que permita visualizar el estado de la tienda en tiempo real o casi real, sin refresco manual.

- Total de productos activos.
- Cantidad de órdenes confirmadas.
- Cantidad de órdenes canceladas.
- Monto total vendido.
- Productos con bajo stock.
- Últimas órdenes registradas.
- Actualización automática al confirmar o cancelar una orden usando SignalR, polling o Server-Sent Events.

5. Aspectos Técnicos

5.1 Arquitectura de Microservicios (NO Monolito)

- La solución debe estar estructurada como microservicios independientes.
- No se acepta una arquitectura monolítica.
- Cada microservicio debe aplicar arquitectura Clean: Dominio, Aplicación, Infraestructura y Presentación/API.

- El frontend debe comunicarse exclusivamente con el API Gateway.
- Los microservicios no deben ser consumidos directamente desde el frontend.
- Cada microservicio debe poder desplegarse como contenedor independiente.

Componente	Responsabilidad mínima
Catálogo Service	Gestión y consulta de productos, stock, categorías y caché de endpoints de lectura.
Órdenes Service	Gestión de carrito/orden, confirmación, cancelación, cálculo de totales, validación de stock, Unit of Work y concurrencia.
API Gateway	Punto único de entrada; enruta peticiones hacia los microservicios usando YARP u Ocelot.
Frontend	Interfaz para catálogo, carrito, confirmación de orden y dashboard.
SQL Server	Persistencia de productos, órdenes, detalles y trazabilidad.

5.2 Uso de Memory Cache de .NET

La solución debe usar las capacidades nativas de caché de .NET para optimizar endpoints de lectura del catálogo. No se requiere instalar servidores externos ni librerías adicionales. Puede utilizarse IMemoryCache o IDistributedCache según criterio del equipo, pero para efectos de la prueba se espera al menos IMemoryCache correctamente configurado.

- Cachear listado de productos activos.
- Cachear detalle de producto.
- Configurar expiración absoluta o deslizante.
- Invalidar caché cuando se cree, edite, inactive o cambie el stock de un producto.
- No usar caché como fuente de verdad para confirmar órdenes.
- La validación final de stock debe hacerse contra SQL Server dentro de una transacción.

5.3 Aplicación del patrón Unit of Work

El microservicio de Órdenes debe aplicar el patrón Unit of Work para coordinar los repositorios y confirmar la transacción de negocio como una unidad atómica.

- Crear orden.
- Crear detalle de orden.
- Validar y descontar stock.
- Calcular totales.
- Confirmar cambios si todo es exitoso.
- Realizar rollback si ocurre cualquier error de negocio, persistencia o concurrencia.

5.4 Control de Concurrencia

La solución debe garantizar que no se venda más stock del disponible. El control debe implementarse en backend y base de datos, no en frontend.

- Escenario obligatorio: Producto A tiene 1 unidad disponible. Dos usuarios intentan comprar 1 unidad al mismo tiempo. Solo una orden debe confirmarse; la otra debe rechazarse por stock insuficiente o conflicto de concurrencia.
- Se recomienda usar transacción con bloqueo pesimista sobre el producto o control optimista con RowVersion.
- La operación debe devolver un error controlado, preferiblemente HTTP 409 Conflict, cuando exista conflicto de concurrencia.
- El stock debe recalcularse o validarse nuevamente dentro de la transacción antes de confirmar la orden.

5.5 Health Check de Base de Datos

- Cada microservicio debe exponer GET /health.
- El health check debe validar la conexión real a SQL Server.
- Respuesta esperada: { status: 'Healthy', database: 'Connected' }.

5.6 APIs REST

Todos los endpoints deben estar documentados mediante Swagger, OpenAPI o Scalar y ser accesibles a través del API Gateway.

Servicio	Endpoint requerido	Descripción
Catálogo	GET /productos	Consultar productos activos.
Catálogo	GET /productos/{id}	Consultar detalle de producto.
Catálogo	POST /productos	Registrar producto.
Catálogo	PUT /productos/{id}	Editar producto.
Órdenes	POST /ordenes	Confirmar orden.
Órdenes	GET /ordenes/{id}	Consultar orden.
Órdenes	DELETE /ordenes/{id}	Cancelar orden lógicamente.
Dashboard	GET /dashboard/resumen	Consultar indicadores de tienda.
Health	GET /health	Validar estado del microservicio y conexión a SQL Server.

5.7 Unit Tests

El microservicio de Órdenes debe contar con un proyecto de unit tests independiente. Los tests deben cubrir las reglas de negocio principales. Se solicitan al menos 10 pruebas: 5 positivas y 5 negativas. Debe incluirse al menos un test de concurrencia.

Tipo	Caso de prueba
Positivo	Confirmar orden con un producto activo y stock suficiente.
Positivo	Confirmar orden con varios productos y cálculo correcto de totales.
Positivo	Cancelar orden confirmada y restaurar stock.
Positivo	Consultar producto desde caché después de primera lectura.
Positivo	Aplicar Unit of Work confirmando orden y detalle en una sola transacción.
Negativo	Rechazar orden vacía.
Negativo	Rechazar producto inactivo.
Negativo	Rechazar cantidad menor o igual a cero.
Negativo	Rechazar orden con stock insuficiente.
Negativo	Simular dos compras concurrentes sobre el último producto disponible y verificar que solo una se complete.

6. Entregables

- Diagrama de arquitectura: microservicios, API Gateway, base de datos, frontend y caché.
- Casos de uso del sistema.
- Prototipo de interfaz de usuario.
- Código fuente de todos los proyectos.
- Scripts de base de datos SQL Server.
- Implementación de Catálogo Service.

- Implementación de Órdenes Service.
- Implementación de API Gateway con YARP u Ocelot.
- Frontend funcional.
- Uso de Memory Cache nativo de .NET en endpoints de catálogo.
- Implementación del patrón Unit of Work en confirmación/cancelación de órdenes.
- Proyecto de pruebas unitarias.
- Evidencia de health checks.
- Evidencia de Swagger/OpenAPI.
- README con instrucciones de ejecución local y despliegue.
- Solución publicada en Azure Container Apps, si el entorno de evaluación lo requiere.

7. Tabla de Evaluación

Tema	Pts.	Aspectos por calificar	Pts. Asig.
Análisis del Requerimiento	10	Presenta diagrama completo del problema: catálogo, carrito, confirmación de orden, descuento de stock y cancelación.	5
		Genera los casos de uso del problema con flujos principales, alternos, excepciones y postcondiciones.	5
Planificación de la solución	10	Elaboración de prototipo y correspondencia clara con el producto finalizado.	10
Capa de presentación ASP.NET MVC, JavaScript, AJAX, jQuery o Angular	15	Utiliza tecnologías permitidas para la capa de presentación.	5
		Cumple con los campos indicados: catálogo, carrito, detalle de orden, totales y dashboard.	5
		Cumple con validaciones indicadas en el requerimiento y actualización sin refresco manual.	5
Microservicios (Container Apps)	40	Diseño apropiado de arquitectura de microservicios. Cumple con arquitectura Clean: Aplicación, Dominio, Infraestructura y Presentación/API.	8
		Realiza correctamente cálculos de subtotal, impuesto, total y stock disponible.	5
		Expone correctamente APIs REST documentadas con	7

		Swagger/OpenAPI/Scalar	
		Acceso a datos SQL Server.	6
		Uso correcto de LINQ, Dapper o Entity Framework.	4
		Uso de Memory Cache nativo de .NET en endpoints de lectura e invalidación adecuada.	5
		Aplicación del patrón Unit of Work en operaciones transaccionales de órdenes e inventario.	5
Buenas prácticas en desarrollo de Software	10	Realización de unit tests en el proyecto, incluyendo casos positivos, negativos y concurrencia.	6
		Código limpio, documentado, manejo de errores y respuestas HTTP adecuadas.	4
Eficiencia y Gestión del Tiempo	15	Entrega menor a 4 horas.	15
		Entrega entre 4 y 6 horas.	10
		Entrega mayor a 6 horas.	5
TOTAL	100		100

8. Reglas para considerar

- Se deberá mantener en todos los equipos únicamente las aplicaciones requeridas para la ejecución de la prueba.
- Se dará un lapso inicial de 20 minutos para lectura y aclaración de dudas.
- No se permite el uso de herramientas de colaboración o canales de comunicación no autorizados.
- No se permite recibir apoyo externo durante la prueba.
- En caso de detectarse colaboración de terceros, la prueba podrá ser anulada.
- La solución debe ser desarrollada exclusivamente por el equipo asignado.

9. Anexo A - Casos de uso mínimos esperados

CU-01 Consultar catálogo de productos

- Actor: Cliente o usuario interno.
- Precondición: existen productos activos.
- Flujo: usuario abre catálogo, frontend solicita productos al API Gateway, Catálogo Service consulta caché o SQL Server, sistema muestra productos.
- Excepciones: catálogo vacío, servicio no disponible, error de base de datos.

CU-02 Agregar producto al carrito

- Actor: Cliente.

- Precondición: producto activo.
- Flujo: usuario selecciona producto y cantidad, sistema valida cantidad mayor a cero, agrega producto al carrito y recalcula subtotal visual.
- Excepciones: cantidad inválida, producto inactivo, producto no encontrado.

CU-03 Confirmar orden

- Actor: Cliente.
- Precondición: carrito con productos.
- Flujo: usuario confirma compra, backend valida productos, stock, precios, calcula totales, descuenta stock y registra orden usando Unit of Work.
- Excepciones: stock insuficiente, conflicto de concurrencia, producto inactivo, error de persistencia.

CU-04 Cancelar orden

- Actor: Usuario administrador.
- Precondición: orden confirmada.
- Flujo: usuario solicita cancelación, sistema valida estado, cambia a Cancelada, restaura stock y registra trazabilidad.
- Excepciones: orden inexistente, orden ya cancelada, error de concurrencia.

CU-05 Consultar dashboard

- Actor: Usuario administrador.
- Precondición: existen productos u órdenes.
- Flujo: usuario ingresa al dashboard, sistema muestra indicadores y actualiza información sin refresco manual.
- Excepciones: error de servicio, error de base de datos.

10. Anexo B - Modelo de datos sugerido

Tabla	Campos sugeridos
Productos	Id, Nombre, Categoria, PrecioUnitario, StockDisponible, Estado, FechaCreacion, RowVersion
Ordenes	Id, Cliente, FechaCreacion, Subtotal, Impuesto, Total, Estado, FechaCancelacion, MotivoCancelacion
OrdenDetalle	Id, OrdenId, ProductId, NombreProducto, PrecioUnitario, Cantidad, SubtotalLinea
AuditoriaStock	Id, ProductId, TipoMovimiento, Cantidad, StockAnterior, StockNuevo, Fecha, Referencia

11. Anexo C - Estrategia recomendada de concurrencia

Para una prueba de 6 horas se recomienda implementar una estrategia simple, demostrable y defendible: transacción con bloqueo pesimista sobre los productos que serán descontados durante la confirmación de la orden. Otra opción válida es usar RowVersion con optimistic locking, siempre que se maneje correctamente el conflicto y se devuelva una respuesta controlada.

1. Iniciar transacción mediante Unit of Work.
2. Leer los productos requeridos aplicando bloqueo o validación de RowVersion.
3. Validar estado activo y stock suficiente para cada producto.
4. Calcular totales en backend.
5. Crear orden y detalle.
6. Descontar stock.

7. Registrar auditoría de stock.
8. Confirmar transacción.
9. Invalidar caché de productos afectados.
10. Si ocurre error, ejecutar rollback y devolver 400, 404 o 409 según corresponda.

12. Checklist de entrega

- ☐ Diagrama de arquitectura incluido.
- ☐ Casos de uso incluidos.
- ☐ Prototipo incluido.
- ☐ Frontend consume solo API Gateway.
- ☐ Catálogo Service implementado.
- ☐ Órdenes Service implementado.
- ☐ API Gateway implementado.
- ☐ SQL Server utilizado.
- ☐ Memory Cache configurado en endpoints de lectura.
- ☐ Caché invalidada cuando cambia producto o stock.
- ☐ Unit of Work aplicado en confirmación/cancelación de órdenes.
- ☐ Concurrencia validada.
- ☐ Health checks funcionando.
- ☐ Swagger/OpenAPI funcionando.
- ☐ Tests unitarios ejecutados.
- ☐ README incluido.
- ☐ Despliegue realizado o documentado.